

Adders

Empezar tarea

- Fecha de entrega Jueves a las 23:59
- Puntos 100
- Entregando una carga de archivo
- Disponible después de 29 de mayo en 0:00

Realizar la implementación de 2 Adders parametrizables de N Bits.

1. Ripple Carry Adder

2. Carry Look Ahead / Parallel Prefix Adder (Escoger alguno de los 2 adders, en caso de elegir Carry Look Ahead la unidad de generación de carry es de 4-bits)

Los adders deben de respetar la siguiente interface

```
module adder#(  
    parameter int WIDTH = 16          // Ancho del adder  
)(  
    input logic [WIDTH-1:0] srca,      // Operando 1  
    input logic [WIDTH-1:0] srcb,      // Operando 2  
    input logic cin,                  // Carry de entrada  
    input logic is_signed,            // Indica si la operacion es signed(1) o unsigned(0)  
    output logic [WIDTH-1:0] result,   // Resultado  
    output logic cout,                // Carry de salida  
    output logic zero_f,              // Bandera de cero  
    output logic ov_f                 // Bandera de overflow  
);
```

Cada adder sintetizarlo utilizando 4 parametros de WIDTH diferentes, 8, 32, 64 y 72.

En el reporte incluir la frecuencia máxima alcanzada por el circuito, así como la ruta crítica, área y niveles de lógica de la ruta crítica

Recuerden revisar la rubrica de calificación para incluir todo lo necesario en el reporte.

Anexo el TB simple del adder con el cual pueden verificar su diseño

[tb_adder.sv \(https://canvas.iteso.mx/courses/56482/files/12243845?wrap=1\)](https://canvas.iteso.mx/courses/56482/files/12243845?wrap=1) ↓
(https://canvas.iteso.mx/courses/56482/files/12243845/download?download_frd=1)

Les comparto los scripts de síntesis que utilicé para mi tesis para los que quieran utilizar la tool de openRoad o las del ITESO

Convert SV to V - https://github.com/miguel100398/RISCV_V/blob/main/scripts/SV2V/sv2v.sh 
(https://github.com/miguel100398/RISCV_V/blob/main/scripts/SV2V/sv2v.sh). (Utiliza esta tool
<https://github.com/zachjs/sv2v> ) (<https://github.com/zachjs/sv2v>)

Yosys - https://github.com/miguel100398/RISCV_V/blob/main/scripts/Yosys/synthesis.tcl 
(https://github.com/miguel100398/RISCV_V/blob/main/scripts/Yosys/synthesis.tcl)

Genus - https://github.com/miguel100398/RISCV_V/tree/main/scripts/genus/input_genus 
(https://github.com/miguel100398/RISCV_V/tree/main/scripts/genus/input_genus)

Innovus - https://github.com/miguel100398/RISCV_V/tree/main/scripts/innovus 
(https://github.com/miguel100398/RISCV_V/tree/main/scripts/innovus)

OpenRoad - https://github.com/miguel100398/RISCV_V/tree/main/scripts/openROAD 
(https://github.com/miguel100398/RISCV_V/tree/main/scripts/openROAD)

Pueden utilizar estos PDKs open source

<https://github.com/google/gf180mcu-pdk>  (<https://github.com/google/gf180mcu-pdk>) - 180nm

<https://github.com/google/skywater-pdk>  (<https://github.com/google/skywater-pdk>) - 130nm

<https://github.com/mflowgen/freepdk-45nm>  (<https://github.com/mflowgen/freepdk-45nm>) - 45nm

<https://asap.asu.edu/>  (<https://asap.asu.edu/>) - 7nm